

Lab 08

Author: Akhash Arjundayal (aarjunda)

Summary

This lab implemented a toaster oven as a software finite state machine, using an event-driven pattern where lightweight ISRs only set flags and `main()` polls those flags to run the state machine. The FSM has four required states (`SETUP`, `SELECTOR_CHANGE_PENDING`, `COOKING`, `RESET_PENDING`) plus an `ALERT` state I added for the extra credit. A 100Hz timer polls buttons, a 5Hz timer drives a free-running counter used to distinguish short and long button presses and to time the cook countdown, and the ADC samples the potentiometer to set the time or temperature depending on mode and a time/temp selector. I thought the most interesting part of the lab, more so than the manual emphasizes, was how many of the state transitions are really just deciding whether a button hold was long enough. `BTN3`'s mode-cycle vs. selector-toggle and `BTN4`'s cook-cancel vs. resume-cooking are both the same short-press/long-press pattern reused twice.

Approach

I worked through this the same way as previous labs: build and verify one piece at a time on hardware before moving on, rather than writing the whole state machine at once. I started with `updateOvenOLED()` using dummy data to get the layout right, then wired it to the real oven struct, then added the `ADC_CHANGED` transition, `BTN3`'s mode/selector logic, `BTN4`'s cooking/countdown/LED-bar logic, and finally the `ALERT` extra credit. The hardest part was the ADC: the potentiometer was correctly wired the whole time, but neither `ADC_CONTINUOUS_WATCHDOG` nor `ADC_CONTINUOUS_INTERRUPT` ever fired their callback on this board, which I confirmed by printing from inside the callback and getting nothing. Switching to `ADC_SINGLE_SHOT_INTERRUPT` with the ISR manually re-triggering each conversion finally worked. Right after, the OLED's I2C bus intermittently wedged, printing "I2C Tx Error" and freezing the buttons/pot; several software fixes didn't help, but a full power cycle cleared it every time, pointing to a genuinely wedged bus rather than my code. If I did this again, I'd write a small ADC/interrupt sanity-check harness before building on top of it, instead of assuming the documented mode would just work.

Results and Feedback

This lab took about 7 hours total, done on my own. Once the ADC and I2C issues were sorted out, the actual state machine logic came together quickly, since the FSM diagram in the manual maps almost directly onto a switch statement. For the extra credit, I added an `ALERT` state entered when cooking finishes naturally: it blinks the display via `OLED_SetDisplayInverted()/OLED_SetDisplayNormal()` on every timer tick and is dismissed by

any button press. Since the only tick source available is the 5Hz TIM2, toggling every tick gives roughly 2.5Hz rather than an exact 2Hz, the closest clean approximation available without dedicating another timer to it. I liked this lab more than I expected to, mainly because debugging the ADC and I2C issues forced me to verify my assumptions on hardware instead of trusting the driver headers, which felt more realistic than earlier labs. The manual gave enough detail to get started on each state without much outside help, and the FSM diagram was clear enough to implement almost directly.